# Preguntas Parcial 2

## Dados los principios de manifiesto ágil, explique como el Framework Scrum 2020 propone aplicar esos principios. Justifique.

1. **Satisfacción del cliente mediante entrega temprana y continua**: Scrum asegura entregas continuas con Sprints de duración fija y la creación de Incrementos al cumplir la Definición de Terminado.
2. **Aceptar cambios en los requisitos**: Scrum fomenta cambios a través de las Revisiones del Sprint, promoviendo la colaboración y retroalimentación del cliente.
3. **Entrega frecuente de software funcional**: Los Sprints (de hasta un mes) permiten entregar software en tiempos cortos y frecuentes.
4. **Colaboración entre negocio y desarrolladores**: Los stakeholders participan con el equipo en las Revisiones del Sprint, apoyados por el Scrum Master.
5. **Individuos motivados**: Los Equipos Scrum son auto-gestionados, decidiendo roles, tiempos y métodos de trabajo de forma independiente.
6. **Comunicación cara a cara**: Scrum promueve la comunicación directa a través de los Daily Scrums, Revisiones del Sprint y la eliminación de barreras por parte del Scrum Master.
7. **El software funcional como medida de progreso**: Cada Sprint entrega Incrementos utilizables y verificados que contribuyen al Objetivo del Producto.
8. **Desarrollo sostenible**: Los ciclos consistentes de Sprint fomentan un ritmo de trabajo constante y sostenible para los equipos.
9. **Enfoque en la excelencia técnica**: Las Retrospectivas del Sprint y los equipos multifuncionales mejoran la calidad y agilidad.
10. **Simplicidad al minimizar trabajo innecesario**: Scrum reduce trabajo innecesario mediante la Definición de Terminado, la refinación del Product Backlog y eventos optimizados.
11. **Diseños emergentes de equipos auto-organizados**: Los Equipos Scrum son auto-gestionados, determinando sus flujos de trabajo de manera autónoma.

12. **Reflexión y ajuste regulares:** Las Retrospectivas del Sprint aseguran que los equipos mejoren continuamente en calidad y efectividad.

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas a la ARQUITECTURA.

- ¿La arquitectura satisface todos los requisitos funcionales y no funcionales documentados?
- ¿La arquitectura es consistente con los estándares de la organización y las mejores prácticas de la industria?
- ¿Están todos los componentes del sistema y sus interfaces claramente definidos y documentados?
- ¿La arquitectura está diseñada para facilitar las actividades de prueba de manera efectiva?
- ¿Se han identificado y abordado adecuadamente todos los riesgos arquitectónicos potenciales?
- ¿La arquitectura ha sido revisada y aprobada por todas las partes interesadas relevantes, incluidos arquitectos, analistas, diseñadores, líderes de proyectos y probadores?
- ¿La arquitectura admite futuras necesidades de mantenimiento, escalabilidad y mejora?

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas al DISEÑO.

1. **Alineación con Requisitos**
    - ¿El diseño satisface todos los requisitos funcionales y no funcionales especificados?
2. **Consistencia del Diseño**
    - ¿Es el diseño consistente en todos los componentes y módulos?
3. **Factibilidad**
    - ¿El diseño es técnicamente factible con la tecnología y recursos actuales?
4. **Modularidad**
    - ¿El diseño promueve la modularidad y encapsulación?

5. **Mantenibilidad**
    - ¿El diseño está estructurado para facilitar el mantenimiento y actualizaciones futuras?
6. **Cumplimiento con Estándares**
    - ¿El diseño cumple con los estándares de la industria y de la organización?
7. **Testabilidad**
    - ¿El diseño permite pruebas efectivas de todos los componentes?

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas a PLANES DE PROYECTO

1. **Claridad del Alcance**
    - ¿El alcance del proyecto está claramente definido y aprobado por todas las partes interesadas?
2. **Asignación de Recursos**
    - ¿Los recursos están adecuadamente asignados para todas las tareas del proyecto?
3. **Gestión de Riesgos**
    - ¿El plan del proyecto incluye una estrategia integral de gestión de riesgos?
4. **Realismo de los Tiempos**
    - ¿Los cronogramas y hitos del proyecto son realistas y alcanzables?
5. **Comunicación con las Partes Interesadas**
    - ¿Existe un plan claro de comunicación para las partes interesadas y los miembros del equipo?
6. **Aseguramiento de Calidad**
    - ¿El plan incluye actividades adecuadas de aseguramiento y control de calidad?
7. **Cumplimiento Regulatorio**

- ¿El plan del proyecto cumple con todas las normativas y requisitos legales relevantes?

---

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas a LA ERS.

1. **Cobertura de Requisitos**
   - ¿Todos los requisitos necesarios están documentados sin omisiones?
2. **Claridad y Ausencia de Ambigüedad**
   - ¿Los requisitos están claramente definidos y sin ambigüedades?
3. **Consistencia**
   - ¿Hay requisitos conflictivos dentro del SRS?
4. **Trazabilidad**
   - ¿Cada requisito puede rastrearse hasta su origen y elementos de diseño?
5. **Factibilidad**
   - ¿Los requisitos son técnica y económicamente factibles?
6. **Es evaluable**
   - ¿Cada requisito está redactado de forma que permita pruebas efectivas?
7. **Aprobación de las Partes Interesadas**
   - ¿Todas las partes interesadas han revisado y aprobado los requisitos?

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas al CÓDIGO FUENTE

1. **Cumplimiento con Estándares de Codificación**
   - ¿El código cumple con los estándares y convenciones definidos?
2. **Implementación de Funcionalidades**
   - ¿El código implementa correctamente la funcionalidad prevista según el diseño?
3. **Legibilidad y Documentación**

- ¿El código está bien documentado y es fácil de entender?

4. **Manejo de Errores**
    - ¿El código maneja correctamente excepciones y condiciones de error?
5. **Optimización de Rendimiento**
    - ¿El código está optimizado para el rendimiento donde es necesario?
6. **Consideraciones de Seguridad**
    - ¿El código aborda posibles vulnerabilidades de seguridad?
7. **Verificación de Pruebas**
    - ¿El código ha sido suficientemente probado con pruebas unitarias u otros métodos?

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas a PLAN DE PRUEBAS

1. **Cobertura de Pruebas**
    - ¿El plan de pruebas cubre todos los requisitos funcionales y no funcionales?
2. **Claridad de los Casos de Prueba**
    - ¿Los casos de prueba están claramente definidos con resultados esperados?
3. **Asignación de Recursos para Pruebas**
    - ¿Los recursos necesarios están asignados para las actividades de prueba?
4. **Alineación del Cronograma**
    - ¿El cronograma de pruebas está alineado con el cronograma general del proyecto?
5. **Proceso de Seguimiento de Defectos**
    - ¿Existe un proceso para rastrear y gestionar defectos?
6. **Cumplimiento con Estándares de Pruebas**
    - ¿El plan de pruebas cumple con los estándares de pruebas de la industria y de la organización?

7. **Estrategias de Pruebas Basadas en Riesgos**
    - ¿El plan incluye estrategias para probar más a fondo las áreas de alto riesgo?

## Diseñe un checklist con al menos 5 preguntas, que se pueda utilizar para realizar revisiones técnicas a CASOS DE USO

1. **Cobertura de requisitos**

- ¿Los casos de uso cubren exhaustivamente todos los requisitos funcionales especificados

2. **Claridad y Ausencia de ambigüedad**

- ¿Los casos de uso están escritos de forma clara e inequívoca, lo que garantiza un entendimiento coherente entre todos los revisores?

3. **Consistencia**

- ¿Los casos de uso son consistentes entre sí y con otra documentación del proyecto, como el SRS y los documentos de diseño?

4. **Identificación del actor**

- ¿Están todos los actores identificados correctamente y representan con precisión los roles de usuario que interactúan con el sistema?

5. **Flujo de eventos**

- ¿Los casos de uso describen claramente los flujos normales y alternativos de eventos sin omisiones?

6. **Factibilidad**

- ¿Los casos de uso son técnicamente factibles dadas las limitaciones actuales de tecnología y recursos?

7. **Es evaluable**

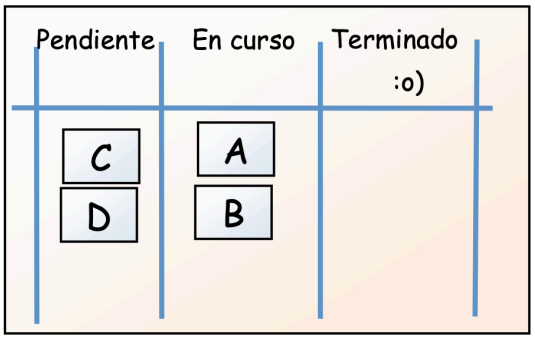
- ¿Se pueden derivar efectivamente los casos de prueba de los casos de uso para validar la funcionalidad del sistema?

## Diferencia entre Scrum y Kanban

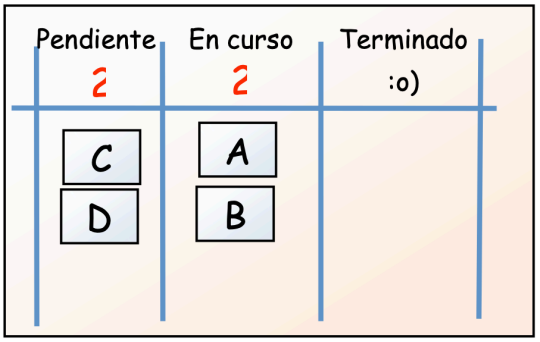
- Scrum es más prescriptivo que Kanban, es decir, hay más reglas a seguir (iteraciones fijas, equipos, etc.)
- Scrum prescribe **roles**: product owner, scrum master y developers.

- Scrum prescribe iteraciones de tiempo **fijo**. En kanban se puede hacer de forma regular, o a demanda.
- Scrum asume compromiso. Kanban es opcional.
- Scrum Métrica Velocidad, Kanban Lead Time y Cycle Time
- Scrum usa gráficos burn-down, kanban no necesariamente
- Las tareas deben poder completarse en un sprint. En Kanban no se define nada
- En scrum el backlog debe ser priorizado, en kanban no necesariamente.
- Scrum prescribe equipos multi-funcionales.  En Kanban, los equipos multi-funcionales son opcionales, y un tablero no necesita estar en manos de un equipo específico. Un tablero está relacionado con un flujo de trabajo, no necesariamente con un equipo.
- Kanban limita el WIP directamente por estado en flujo de trabajo, Scrum limita el WIP indirectamente por iteración. En tableros scrum no se limita el WIP, pero sí existe un límite implícito ya que la iteración en sí tiene un alcance fijo.
- Scrum se resiste a los cambios durante la iteración. Ejemplo:

  ¿Qué ocurre si alguien cambia de idea y quiere añadir E al tablero?




Tablero Scrum




Tablero Kanban

En Scrum diría “no, ya estamos comprometidos a A, B, C y D. Podremos agregar E si el PO lo considera para la próxima iteración.”

En Kanban diría “agregalo a la cola de producto, pero el WIP es 2 y ya hay 2 elementos, por lo que deberías sacar a C o D. Ahora estamos trabajando con A y B, así

que cuando terminemos trabajaremos en los pendientes".

- En Scrum El tablero sprint se limpia entre iteraciones, en Kanban por lo general no.
- Scrum prescribe la estimación y la velocidad. Kanban no.

## Similitudes entre Scrum y Kanban

- Ambos son Lean y Ágiles.
- Ambos emplean sistemas de planificación "pull".
- Ambos establecen límites WIP (scrum indirecto kanban directo).
- En ambos la visibilidad del proceso es la base de su mejora.
- Ambos tienen como objetivo la entrega temprana y frecuente de software.
- Ambos trabajan con equipos auto-organizados.
- Ambos necesitan la división del trabajo en partes.
- Ambos revisan y mejoran de forma continua el plan del producto en base a datos empíricos (velocidad / tiempo de entrega)

## Consideraciones de contexto para el desarrollo de la aplicación móvil que permitirá escuchar música desde el celular:

El equipo de trabajo será el siguiente:

- 1 Analista Funcional
- 3 Desarrolladores
- 1 Diseñador de UX
- 2 Analistas de Prueba
- 1 Responsable de Despliegue

Se definieron los siguientes acuerdos:

- El referente del cliente validará el diseño de UX y UI antes de que se integre a la funcionalidad.
- Se realizarán revisiones técnicas al código como parte del Criterio de Hecho.

a. **Diseñe el tablero completo, considerando las condiciones de contexto y aplicando las prácticas de Kanban. (20 puntos)**

b. **Defina los tipos de trabajo que utilizará, justificando la respuesta y al menos 5 políticas de calidad. (10 puntos)**

a. Tablero de Kanban con las columnas:

- Cola de producto (WIP=5)
- Análisis: En progreso y hecho (WIP=4)
- Diseño: En progreso y hecho (WIP=2)
- Desarrollo: En progreso y hecho (WIP=2)
- Revisión: Por revisar y revisada (WIP=3)
- Validación: Por validar y validada
- En producción

**b. Tipos de Trabajo:**

1. **Desarrollo de Funcionalidades:** Creación de nuevas características para la aplicación, como listas de reproducción o búsquedas.
    - Justificación: Es el núcleo del producto, necesario para satisfacer las necesidades del cliente y usuarios.
2. **Corrección de Errores:** Solución de problemas identificados en el código o en pruebas.
    - Justificación: Garantiza la estabilidad y funcionalidad de la aplicación.
3. **Pruebas y Validaciones:** Actividades de testeo funcional y técnico.
    - Justificación: Asegura la calidad antes de entregar al cliente.
4. **Diseño UX/UI:** Creación y refinamiento de interfaces de usuario.
    - Justificación: Mejora la experiencia del usuario final y satisface los criterios del cliente.
5. **Despliegue y Configuración:** Preparación y lanzamiento en entornos de producción.
    - Justificación: Permite que las funcionalidades lleguen al usuario final de forma estable.

**Políticas de Calidad:**

1. **Reposición de Trabajo en el Tablero:**
    - **Cuándo:** Cada vez que una columna tenga menos ítems que el límite mínimo de trabajo (WiP).
    - **Cantidad:** Suficiente para mantener el flujo continuo, respetando los límites de WiP.
    - **Por Quién:** El equipo completo durante reuniones de planificación.
2. **Definición de Actividad Terminada (Criterios de Tracción):**
    - **Criterios:**
        - Código revisado y aprobado por otro desarrollador.
        - UX/UI validado por el cliente.
        - Pruebas superadas sin errores críticos.
    - **Condición:** Solo puede moverse al siguiente estado si cumple con los criterios definidos.
3. **Límites de Trabajo en Curso (WiP):**
    - **Límites por Columna:**
        - Desarrollo: Máximo 3 ítems.
        - Pruebas: Máximo 2 ítems.
        - Despliegue: Máximo 1 ítem.
    - Justificación: Evita sobrecarga y asegura un flujo continuo.
4. **Gestión de Clases de Servicio:**
    - **Políticas:**
        - Ítems "Expreso" se priorizan automáticamente.
        - Fecha Fija tiene prioridad secundaria.
        - Estándar se gestiona según capacidad.
        - Intangibles se manejan si hay capacidad ociosa.
    - Justificación: Garantiza equilibrio entre urgencias y tareas estratégicas.
5. **Cadencia de Reuniones:**
    - **Planificación semanal:** Lunes por la mañana (revisión y reposición de tareas).

- **Revisión diaria:** 15 minutos al inicio del día para ajustar el flujo.
- **Revisión quincenal:** Evaluación de progreso con el cliente.
- Justificación: Mantiene alineación constante y detecta bloqueos rápidamente.

## Nexus

**Ud. tiene que presentar una propuesta en respuesta a un requerimiento de un cliente que ha manifestado la necesidad de tener una versión de un producto de software para poner en producción en una fecha cercana para lo cual está dispuesto a invertir para tener 3 equipos trabajando en paralelo; por tanto se le ha requerido que presente una propuesta de cómo estructuraría el equipo de proyecto utilizando un framework para escalar Scrum.**

Es básicamente la guía de Nexus

## Métricas

**Dado el dominio de las métricas de software en la gestión tradicional, desarrolle una situación de ejemplo en la cual ud. Sea el Lider de Proyecto y describa un proyecto de desarrollo de software y elabore para ese contexto un ejemplo propio de una métrica para cada dominio (proyecto, proceso, producto) y explique para que usarla.**

Ejemplo: Desarrollo de un sistema de gestión de inventarios para una cadena de tiendas minoristas.

**Métrica del Proyecto**: **Progreso porcentual frente al cronograma**

- **Ejemplo**: Si el cronograma establece que el proyecto debería estar al 50% al mes 3, pero solo se ha completado un 40%, el progreso real es un 80% del esperado.
- **Uso**: Identificar retrasos o adelantos en el cronograma y ajustar recursos o tareas para cumplir plazos.

**Métrica del Producto**: **Índice de estabilidad de requisitos**

- **Ejemplo**: Si se han definido 100 requisitos al inicio y solo 10 se han modificado durante el desarrollo, el índice es 90%.
- **Uso**: Evaluar la estabilidad de los requisitos y el impacto de los cambios en el alcance del producto.

**Métrica del Proceso**: **Cumplimiento de hitos internos**

- **Ejemplo**: Si se planificaron 5 hitos en una etapa del proyecto (como diseño, desarrollo y pruebas iniciales) y solo se lograron 4 dentro del tiempo previsto, el cumplimiento es del 80%.
- **Uso**: Analizar la efectividad del proceso de gestión y tomar medidas para evitar retrasos futuros.

## Propósito, características y diferencias entre testing y auditorías

| Aspecto | Testing de Software | Auditorías de Software |
|---|---|---|
| **Propósito** | Identificar defectos en el software para mejorar su calidad y reducir riesgos antes de su liberación. | Evaluar la conformidad de productos o procesos con estándares, lineamientos y requisitos establecidos. |
| **Naturaleza** | Destructiva: busca encontrar errores y defectos en el sistema. | Constructiva: asegura el cumplimiento de procesos, estándares y la mejora continua. |
| **Objetivo** | Verificar y validar que el software cumpla con los requisitos funcionales y no funcionales. | Asegurar la calidad en el proceso y producto final, proporcionando una opinión objetiva e independiente. |
| **Enfoque** | Se enfoca en la detección de errores a través de la ejecución de casos de prueba diseñados. | Se enfoca en evaluar si los procesos y productos cumplen con estándares predefinidos. |
| **Momento de Aplicación** | Durante todo el ciclo de vida del desarrollo del software, especialmente en fases de implementación y pruebas. | Usualmente realizado durante o al final de fases críticas del proyecto o del desarrollo del software. |
| **Resultados** | - Identificación de defectos.<br>- Informes de fallas encontradas.<br>- Mejora del producto antes de la entrega. | - Informes de cumplimiento.<br>- Observaciones, desviaciones y buenas prácticas.<br>- Planes de acción para resolver no conformidades. |
| **Responsable** | Realizado por testers y, en casos específicos, por desarrolladores (pruebas unitarias). | Realizado por auditores independientes del equipo de desarrollo. |
| **Ejemplo de Técnicas** | - Testing de Caja Negra y Caja Blanca.<br>- Pruebas funcionales | - Auditorías de Proyecto.<br>- Auditorías de Configuración |

| Aspecto | Testing de Software | Auditorías de Software |
|---|---|---|
| | y no funcionales.<br>- Testing unitario, integración, sistema y aceptación. | Funcional y Física.<br>- Revisión de registros y muestreo. |
| **Costo** | Representa entre el 30-50% del costo total del desarrollo para garantizar software confiable. | Implica esfuerzo adicional, pero asegura calidad y detecta riesgos en etapas tempranas, lo que puede ahorrar costos a largo plazo. |
| **Resultado Esperado** | Un software funcional y confiable que cumpla con las expectativas del usuario y los requisitos establecidos. | Procesos mejorados y productos conformes que cumplen estándares, fomentando confianza en clientes y stakeholders. |
| **Impacto** | Directamente en la funcionalidad y experiencia del usuario final. | Directamente en la calidad y conformidad del desarrollo del proyecto, lo que influye en la confianza del cliente y el cumplimiento de normativas. |
| **Relación con la Calidad** | Testing no garantiza la calidad del software, pero contribuye a identificar defectos que afectan la misma. | Auditorías aseguran calidad al verificar el cumplimiento de estándares y procedimientos. |

Ambas prácticas son complementarias, con el testing centrado en encontrar defectos en el producto y las auditorías enfocadas en la calidad del proceso y su alineación con estándares.

## Realice una análisis de los principios Lean y explique las prácticas que propone Kanban para aplicar en forma concreta los principios.

También hay q analizar los principios

Kanban propone **6 prácticas concretas** para aplicar los **7 principios Lean** en trabajos del conocimiento. Estas prácticas son:

1. **Visualizar el trabajo**:
    - Muestra el flujo de trabajo y sus riesgos, apoyando la eliminación de desperdicios al hacer visibles problemas como cuellos de botella o trabajo innecesario.

- Facilita decisiones colaborativas, mejorando el aprendizaje continuo.

2. **Limitar el trabajo en curso (WiP)**:
    - Controla el flujo de trabajo asegurando que la capacidad disponible se use de forma eficiente, eliminando desperdicios como esperas y cambios de contexto.
    - Promueve entregas rápidas y calidad en el flujo.
3. **Gestionar el flujo de trabajo**:
    - Optimiza la entrega fluida y predecible, asegurando que las tareas fluyan sin interrupciones innecesarias.
    - Refuerza el principio de "ver el todo" al mejorar la entrega de valor global.
4. **Hacer las políticas explícitas**:
    - Define criterios claros y visibles para todos, promoviendo coherencia, calidad y compromiso informado en los equipos.
5. **Establecer ciclos de retroalimentación**:
    - Permite la mejora continua y el aprendizaje compartido, fomentando el principio de amplificar aprendizaje y empoderar al equipo.
6. **Mejorar colaborativamente y evolucionar experimentalmente**:
    - Utiliza experimentos controlados para implementar mejoras, alineándose con la filosofía de evitar compromisos prematuros y trabajar en calidad.